

CityMind

Intelligent City Management System

Phase 1 Design Document

Deadline: 26 April

24i-0500 (Ammar Bin Omer)
24i-0643 (Haris Zahid Abbasi)
24i-0880 (Hassaan Amin)

1. Introduction

Cities grow faster than their infrastructure. Emergency response gets slower, planning gets harder, and the people making decisions end up working with incomplete information. CityMind is a system built to help city authorities deal with five core operational problems using AI and search techniques.

The city is modeled as a grid-based graph. Each node is a location with a type (Residential, Hospital, School, Industrial, Power Plant, or Ambulance Depot), a population density, a risk index, and an accessibility flag. Edges are roads, each with a travel cost. Standard roads cost 1.0, roads through residential zones cost 0.8, and blocked roads are treated as impassable by all parts of the system.

Key design principle: There is one shared city graph. No module keeps its own copy. If a road floods, every component sees that change straight away.

2. Our Understanding of the Five Challenges

Challenge 1: City Layout Planning

Technique: *Constraint Satisfaction Problem (CSP)*

We need to place different types of locations on an empty grid while following a set of rules:

- Industrial zones cannot be next to schools or hospitals
- Every residential area must be within three road hops of a hospital
- Power plants must be within two road hops of at least one industrial zone

- If no valid layout is possible, the system should identify which rule is causing the conflict and find the layout with the fewest violations

Problem type: Assigning location types to grid cells under hard rules. We are looking for any valid assignment, or the closest one when a fully valid layout does not exist.

Challenge 2: Road Network Optimization

Technique: *Minimum Spanning Tree (Kruskal's Algorithm)*

Once the city layout is done, we need to build roads that connect every location at the lowest total cost. On top of that, there is a safety requirement: the Primary Hospital and the Ambulance Depot must always have at least two completely separate routes between them. If one road goes down, the other route must still work.

Problem type: Finding a connected spanning structure at minimum cost, with a redundancy guarantee for one specific pair of nodes.

Challenge 3: Ambulance Placement

Technique: *Genetic Algorithm*

Three ambulances need to be placed so that the worst-case response time is as short as possible. This means minimizing the maximum distance from any citizen node to the nearest ambulance. The number of possible placements grows very fast with grid size, so trying every combination is not practical.

Problem type: A minimax search problem. We need an efficient method that finds good placements without going through every possibility.

Challenge 4: Emergency Routing Under Changing Conditions

Technique: *A* Search*

A medical team needs to visit a series of trapped civilians in a set order. The city is not static: roads can flood while the team is already moving. When a road gets blocked, the system must immediately recalculate the best available route. The new route has to be the shortest possible given the current state of the graph.

Problem type: Shortest-path search on a graph that can change at any time. The heuristic must be admissible (never overestimate) to guarantee the shortest path. The algorithm also needs to respond to graph changes in real time.

Challenge 5: Crime Risk Prediction and Integration

Technique: *K-Means + Decision Tree*

This challenge has three parts:

- Group neighborhoods into clusters based on population density and how close they are to industrial zones, without using any labeled data
- Generate a synthetic crime dataset from the city graph and train a classifier to predict the risk level (High, Medium, or Low) for each neighborhood
- Feed the predictions back into the shared city graph as cost multipliers so that high-risk areas increase the effective travel cost used by pathfinding and ambulance placement

Problem type: Unsupervised clustering followed by supervised classification, with results feeding directly back into the shared graph used by all other modules.

3. Algorithm Choices and Justifications

Challenge 1: CSP with Backtracking and Forward Checking

The layout problem maps directly to constraint satisfaction. Each grid cell is a variable. The domain of each variable is the set of possible location types. The constraints come straight from the placement rules. Backtracking search tries assignments and backs up whenever a constraint is broken, so it does not waste time going further down a dead end.

We also add forward checking: after each assignment, we immediately remove values from neighboring cells that would break a constraint. This trims the search tree early and avoids reaching a dead end only after many steps.

Why not a greedy approach?

A greedy approach places locations one by one using a priority rule. The problem is that early decisions can lock you into a situation where no valid placement is left for later cells, and there is no way to go back. Greedy placement does not guarantee completeness, meaning it can fail to find a valid layout even when one exists.

Why CSP is better here: Backtracking with forward checking is complete. If a valid layout exists, it will find one. If no valid layout exists, we fall back to a min-conflicts repair strategy that starts from a random assignment and fixes the most-violated constraint step by step. This directly satisfies the requirement to identify the conflicting rule and propose the closest valid solution.

Challenge 2: Kruskal's Algorithm (MST)

A Minimum Spanning Tree connects all nodes at the lowest total edge cost. Kruskal's algorithm does this by sorting edges by cost and adding them one at a time, skipping any edge that would create a cycle. The result is a connected road network at minimum cost.

An MST on its own only gives one path between any two nodes. Since the hospital and the depot need at least two separate routes, we check the path between them after building the MST and add one extra cheap edge to create a second independent route. This keeps the total cost low while satisfying the safety requirement.

Why not a genetic algorithm for network design?

Encoding the redundancy constraint into a fitness function is not straightforward, and there is no guarantee a GA will converge within any set time. For a well-defined problem with a clear objective and a small number of structural constraints, an exact method is simply more dependable since that is quite literally what it was designed for.

Why Kruskal's is better here: MSTs are built specifically for minimum-cost network connectivity. The core problem here is exactly what they solve. The redundancy requirement is handled cleanly by adding one extra edge after the tree is built. In short, since it handles the problem much more effectively, we chose this approach.

Challenge 3: Genetic Algorithm for Ambulance Placement

Each individual in the population is a candidate set of three ambulance positions. The fitness function is based on the inverse of the maximum distance from any citizen node to its nearest ambulance. The algorithm uses selection, crossover (swapping positions between two parent placements), and mutation (randomly moving one ambulance to a new location) across many generations to move toward better solutions.

Why not hill climbing?

A basic local search would start from a random placement and shift each ambulance to a better neighboring position. The issue is that the minimax objective has a lot of flat regions and local minima, so pure hill climbing often gets stuck before finding a good solution.

Why the GA is better here: A genetic algorithm keeps a whole population of candidate solutions and explores many parts of the search space at once. Crossover lets it escape local optima in a way that hill climbing cannot. The problem statement specifically points toward evolutionary search: 'Consider the role of randomness and evolution in search.' Since our main goal is "best" possible solution, and not a "reasonable" solution, we avoid local searches here!

Challenge 4: A* Search

A* finds the shortest path between two nodes using a heuristic to guide the search toward the goal. We use Manhattan distance as the heuristic. On a grid, Manhattan distance never

overestimates the true cost, which makes it admissible and guarantees that A* always returns the shortest path.

When a road gets blocked mid-journey, A* is run again from the team's current position using the updated shared graph. Visiting each civilian is handled as a sequence of separate point-to-point A* queries.

Why not BFS or Dijkstra's?

BFS ignores edge weights entirely. Dijkstra's is correct and complete but has no sense of direction. It's just a greedy algorithm. A* with an admissible heuristic runs faster than Dijkstra's because it skips nodes that are clearly going the wrong way. For a system that needs to replan quickly after a road change, that speed matters.

Key property: Admissibility guarantees that A* always finds the shortest path. This is the one property the heuristic must have for the routing guarantee to hold.

Challenge 5: K-Means Clustering + Decision Tree Classifier

The pipeline runs in four steps:

- Step 1 (Clustering): K-Means groups neighborhoods by population density and distance to the nearest industrial zone. We use $K=3$ to match the three risk levels. No labels are needed, so this is unsupervised learning.
- Step 2 (Data generation): We assign synthetic crime incident rates to each neighborhood using population density, industrial proximity, and cluster membership as inputs. This produces a labeled dataset.
- Step 3 (Classification): A Decision Tree is trained on the synthetic data. It learns which combinations of features map to each risk level and produces rules that city authorities can read and check.
- Step 4 (Integration): The predicted risk level for each node becomes a cost multiplier (High = $1.5x$, Medium = $1.2x$, Low = $1.0x$) that updates edge weights in the shared graph, which immediately changes the costs used by A* and the GA.

Why not a neural network?

A neural network might get higher accuracy, but it is a black box. A decision tree produces rules that are easy to read and explain. For a system that is actually informing decisions made by city staff, being able to explain why a neighborhood is flagged as high risk is just as important as accuracy.

Conceptual note: Step 1 is unsupervised because no pre-labeled crime data exists. Step 3 is supervised because we use the generated labels to train a classifier. This is the core distinction the problem is asking us to understand.

4. Shared City Graph Structure

The city graph is a single shared object that all five modules read from and write to. It is the only source of truth about the current state of the city.

Node Structure

Field	Type	Description
node_id	int	Unique identifier (row * cols + col for grid)
location_type	enum	Residential, Hospital, School, Industrial, Power Plant, Ambulance Depot
population_density	float	Number of people at this location
risk_index	float	Updated by Challenge 5. Starts at 1.0
accessible	bool	Set to False if this location is blocked
cluster_id	int	Assigned by K-Means in Challenge 5
risk_level	enum	High, Medium, or Low (from the classifier)

Edge Structure

Field	Type	Description
from_node	int	Source node ID
to_node	int	Destination node ID
base_cost	float	1.0 for standard roads, 0.8 through residential zones
blocked	bool	Set to True if this road is currently impassable
effective_cost	float	base_cost x risk_multiplier of the destination node

Note: effective_cost is recalculated whenever a road is blocked or a node's risk level changes. All modules use effective_cost for travel calculations.

Dynamic Road Blocking

When a road floods during the simulation:

- The blocked flag on that edge is set to True in the shared graph
- Challenge 4 picks this up straight away (it holds a reference to the graph), drops the current path, and re-runs A*
- Challenge 3 can re-check ambulance positions at the next simulation step using the updated graph

How Each Module Interacts with the Graph

Every module works on the same graph object. There is no copying or syncing between modules. The table below shows what each module reads and writes, and when it runs.

Module	Reads from graph	Writes to graph	When it runs
Challenge 1 (CSP Layout)	Node domains, constraint rules	location_type on each node	Once at setup, before anything else.
Challenge 2 (MST Roads)	node_id, location_type	Edges: from_node, to_node, base_cost, blocked=False	Once, after layout is set.
Challenge 3 (GA Ambulance)	effective_cost on edges, location_type, population_density	Does not write back to graph directly	At setup, then re-runs if effective_cost changes.
Challenge 4 (A* Routing)	effective_cost, blocked flag on edges	Does not write back to graph (read-only)	Every simulation step. Re-runs immediately when a road is blocked.
Challenge 5 (Crime ML)	population_density, location_type, edges for distance calc	risk_index, cluster_id, risk_level on nodes; effective_cost on edges	After roads are built. Re-runs each step to refresh risk data.

Execution order: Challenge 1 runs first (sets layout), then Challenge 2 (builds roads), then Challenge 5 (computes risk scores and updates effective_cost), then Challenges 3 and 4 run during the simulation loop using the fully populated graph.

5. User Interface Wireframe

Below is a rough idea of what our project will look like. We started working on it early and have made significant progress ;however, these can be changed in the future if we feel like we have a better design in mind.

Residential

Hospital

School

Industrial

PowerPlant

AmbulanceDepot

MST Road

Ambulance

Medical Team

Civilian



6. Work Breakdown

Member	Responsibilities
Ammar bin Omer	Challenge 1,3 : CSP layout solver, constraint checking, minimum-conflict fallback and Genetic Algorithm
Hassaan Amin	Challenge 2: Kruskal's MST implementation, redundancy augmentation logic
Haris Zahid Abbasi	Challenge 4: A* search, dynamic replanning on road changes, multi-civilian sequencing and Challenge 5: K-Means clustering, synthetic data generation, Decision Tree classifier, graph integration
All Members	Shared city graph design, UI, 20-step simulation loop, debugging and testing.

Shared responsibility: The shared city graph is owned by everyone on the team. Any changes to its structure need to be agreed on and coordinated across all members before being implemented.

7. Summary Table

Challenge	Technique	Course Coverage
1. Layout Planning	CSP with Backtracking + Min-Conflicts	Lectures 14-15: CSPs
2. Road Network	Kruskal's Algorithm (MST)	DSA (MSTs, permitted with justification)
3. Ambulance Placement	Genetic Algorithm	Lectures 12-13: Genetic Algorithms
4. Emergency Routing	A* Search	Lectures 7-8-9: Informed Searches
5. Crime Prediction	K-Means + Decision Tree	Lectures 10-11: Local Searches / ML Integration
